# Wireless-DFS: Distributed File System

## Team Members

| ID | Name | Sec | BN |
|---|---|---|---|
| 9210684 | Ali Mohamed Farid | 1 | 25 |
| 9213035 | Waleed Ousama Khamees | 2 | 30 |
| 9213347 | Mohamed Maher Hasan Amin | 2 | 12 |
| 9211164 | Mostafa Elsayed | 2 | 20 |

## Introduction

Wireless-DFS is a distributed file system that enables file storage across network nodes. It leverages gRPC for control messaging and uses TCP for file transfers. This ensures both efficiency and resilience in distributed environments.

## Code Structure

The project is organized into several key directories and files:

- **dfs/**
  - Contains the generated Go code for protobuf.
- **datakeeper/**
  - `main.go`: Initializes the DataKeeper service, validates its configuration, and starts the gRPC server to handle file storage and replication.
  - `server.go`: Implements the core logic for the DataKeeper service, including methods for handling file uploads, downloads, and
    replication.

- **client/**
  - `main.go`: Implements the client-side logic for interacting with the distributed file system, including a terminal-based UI for uploading, downloading, and listing files.
- **master/**
  - `main.go`: Initializes the MasterTracker service, starts the gRPC server, and periodically updates the status of DataKeepers.
  - `server.go`: Implements the core logic for the MasterTracker service, including methods for coordinating file uploads, downloads, and replication.
  - `db.go`: Has database initialization logic and table definitions.
- **utils/**
  - `hash.go`: Provides utility functions, such as `HashFileSHA256`, computes the hash of a file to prevent data duplication.

# Key Components

## Protocol Buffers and gRPC

The `dfs.proto` file defines the gRPC services and messages used in the system. Below is a detailed explanation of each service and its RPC methods:

- **MasterTracker Service:**
  - `requestUpload`: Clients use this method to request file uploads. It returns the IP and port of the DataKeeper to upload the file to. The client can then upload the file to this DataKeeper.
  - `requestDownload`: Clients use this method to request file downloads. It returns the IPs and ports of the DataKeepers storing the file. The client can then download the file from any of these DataKeepers.
  - `heartbeat`: DataKeepers use this method to send periodic heartbeats to the MasterTracker. The Request contains the IP, ID, and port of the DataKeeper.

  - `notifyFileStored`: DataKeepers use this method to notify the MasterTracker when a file has been successfully stored.
  - `ListFiles`: Clients use this method to list all available files in the system.

- **DataKeeper Service:**
  - `replicateFile`: Master uses this method to request file replication. The request sent to the DataKeeper contains the file name and the IPs and ports of the DataKeepers to replicate the file to.
  - `requestUpload`: Master uses this method to request file uploads. It opens a random port on the DataKeeper and returns the IP and port to the Master. Also it's used by other DataKeepers to open a port for file replication.
  - `requestDownload`: Master uses this method to request file downloads. It returns the IP and port of the DataKeeper storing the file.

## Client Application

This is the client application that interacts with the DFS.

- `main.go`: It provides a terminal-based UI for uploading, downloading, and listing files. There is also provides a command-line interface for interacting with the DFS.
- **Usage:**
  - `./client upload <file_path>`: Uploads a file to the DFS.
  - `./client download <file_name>`: Downloads a file from the DFS.
  - `./client list`: Lists all available files in the DFS.